

Modélisation et simulation distribuée du mouvement d'un banc de poissons

Rapport du 1er semestre - M1 CHPS

Yohann FRONT-REIGNIER Iram MADANI-FOUATIH Laurence HUANG

3 janvier 2026

Encadrant : Soufian BEN AMOR

Partie 1 - Version séquentielle

Les 3 règles de Reynolds

Architecture logicielle, optimisation et benchmarks

Détails d'implémentation

Partie 2 - Perspectives version parallèle

Motivations

Stratégies de parallélisation

Planning et Livrables S2

Partie 1 - Version séquentielle

- Simuler les comportements collectifs émergents
- Bancs de poissons, flocks d'oiseaux
- 3 règles locales simples appliquées à chaque agent
- **Objectifs S1** : implémentation séquentielle + visualisation 2D
- **Semestre 2** : parallélisation OpenMP/MPI

1. **Séparation** : Éviter les collisions avec les voisins proches
2. **Alignement** : S'adapter à la vitesse moyenne du groupe local
3. **Cohésion** : Se diriger vers le centre de masse des voisins

Appliquées localement $\Rightarrow$ comportement global complexe et réaliste

Formulation Mathématique

Pour chaque boid b , on calcule trois forces :

Séparation :

$$\vec{F}_{sep} = - \sum_{i \in N(b)} \frac{\vec{p}_b - \vec{p}_i}{\|\vec{p}_b - \vec{p}_i\|^2}$$

Alignement :

$$\vec{F}_{ali} = \frac{1}{|N(b)|} \sum_{i \in N(b)} \vec{v}_i - \vec{v}_b$$

Cohésion :

$$\vec{F}_{coh} = \frac{1}{|N(b)|} \sum_{i \in N(b)} \vec{p}_i - \vec{p}_b$$

Mise à jour : $\vec{a} = \vec{F}_{total}/m$, $\vec{v} \leftarrow \vec{v} + \vec{a}\Delta t$, $\vec{p} \leftarrow \vec{p} + \vec{v}\Delta t$

- **Vector2D** : Opérations vectorielles (position, vitesse, accélération)
- **Boid** : Agent autonome avec les 3 règles Reynolds
- **Flock** : Gestionnaire de collection de boids
- **SpatialGrid** : Grille d'accélération pour recherche de voisins
- **GUI (SFML)** : Interface interactive avec sliders temps réel

1. Le Problème

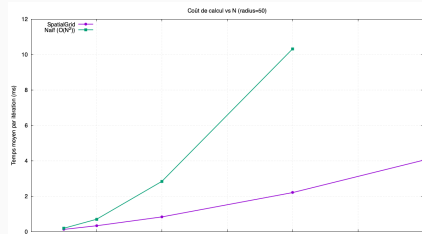
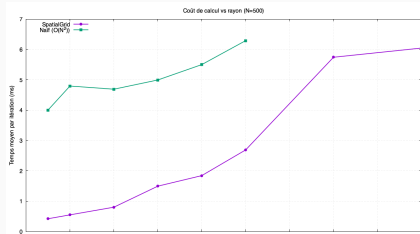
Recherche de voisins naïve = $O(n^2)$ par frame. **Trop lent.**

2. La Solution : Partitionnement

- Partitionner l'espace en cellules = $O(k)$ (où k = boids/cellule)
- **Implémentation** : Grille 2D de vecteurs Boid*
- **Gain** : 500 boids $\rightarrow$ 5x plus rapide (50k comparaisons vs 250k)

Benchmarks de performance

- **Temps vs rayon ($N = 500$)** : le SpatialGrid reste nettement plus rapide que l'algorithme naïf, et l'écart se creuse lorsque le rayon de voisinage augmente.
- **Temps vs nombre de boids (rayon = 50 px)** : la courbe naïve suit une croissance quadratique en N , tandis que le SpatialGrid se rapproche d'une croissance quasi linéaire.
- Benchmarks obtenus en moyennant le temps par itération sur plusieurs centaines de frames, afin de lisser les fluctuations et comparer clairement les deux approches.



- **Langage** : C++20 (moderne, performant)
- **Build** : CMake 3.20+, GCC/Clang avec flags -O3
- **Graphisme** : SFML 2.6.1 pour visualisation 2D
- **Tests** : Google Test, CTest automatique
- **Documentation** : Doxygen
- **Git** : Gestion de version (97 commits)

- **Cohésion** : Agents restent groupés, structure maintenue
- **Alignement** : Direction moyenne émerge rapidement
- **Séparation** : Collisions évitées efficacement
- **Patterns** : Tourbillons, colonnes, vagues observés
- **Performance** : **60 FPS stable** avec 500 boids

- Sliders temps réel : Séparation, Alignement, Cohésion (0-10)
- Rayon de voisinage : 5-200 pixels
- Nombre de boids : 10-500
- Bouton *Apply* pour réinitialiser la simulation
- Affichage des FPS

Contributions de l'équipe

Yohann

(36 commits)

- Vector2D
- Architecture
- Rapport
- Doc Doxygen

Iram

(38 commits)

- Boid (Règles)
- GUI SFML
- Tests Boid

Laurence

(8 commits)

- Flock
- SpatialGrid
- Optimisations
- Benchmarks

Tests Unitaires (Google Test) :

- `test_vector.bin` : Opérations, magnitude, normalisation
- `test_boid.bin` : Forces, mise à jour, règles Reynolds
- `test_flock.bin` : Gestion collection, SpatialGrid
- `test_ui.bin` : Intégration GUI

Validation :

- Validation visuelle des 3 règles émergentes
- Tous les tests passent (100% succès)

Partie 2 - Perspectives version parallèle

Pourquoi paralléliser ?

1. Fidélité à la réalité

- Dans la nature : chaque poisson agit **simultanément**
- Séquentiel : traitement un par un $\rightarrow$ s'éloigne de la réalité
- Parallèle/Distribué : calculs simultanés + synchronisation

2. Complexité algorithmique

- Recherche de voisins : $O(n * k)$ par frame ($k = \text{voisins/cellule}$)
- Pour $n = 10000$ boids à 60 FPS : calculs intensifs
- Limites du séquentiel : quelques centaines de boids max

Limites actuelles :

- ~ 500 boids @ 60 FPS
- SpatialGrid aide mais limité
- Simulations réalistes = 1000-10000+ agents

Objectifs parallélisation :

- 10000+ boids temps réel
- Maintenir 60 FPS
- Exploiter architectures modernes

Speedup attendu :

Approche	Gain
OpenMP (8 coeurs)	Speedup 4-6x
MPI (4 noeuds)	Speedup 8-15x
Hybride	Passage à l'échelle

A valider par benchmarks au semestre 2

Phase 1 : OpenMP (Mémoire Partagée)

Approche : Parallélisation sur multi-coeurs (CPU)

1. Parallélisation de la boucle principale

- Boucle parallèle sur mise à jour des boids
- Chaque thread traite un sous-ensemble de boids
- SpatialGrid partagé en lecture

2. Parallélisation du SpatialGrid

- Construction parallèle de la grille
- Recherche de voisins concurrente
- Attention aux accès concurrents

Complexité : Faible - Bon point de départ - Speedup linéaire

Phase 2 : MPI (Mémoire Distribuée)

Approche : Distribution sur plusieurs machines/noeud

Décomposition de domaine

- Partitionnement spatial de la zone de simulation
- Chaque processus MPI gère une région
- Boids distribués selon leur position

Communication aux frontières

- Echange des boids proches des frontières
- Synchronisation
- Migration des boids entre processus si changement de région

Défi : Equilibrage de charge + minimiser communications

1. Load balancing

- Rééquilibrage si distribution inégale
- Migration de boid entre processus

2. Approche Hybride OpenMP + MPI

- MPI entre noeuds
- OpenMP au sein d'un noeuds
- Exploitation de l'architecture

3. Optimisation Mémoire

- Structure of Arrays au lieu d'un Array of Structures
- Meilleur localité pour le cache

Phase	Période	Objectifs
Phase 1	Jan - Fév	OpenMP + tests de performances
Phase 2	Fév - Mars	MPI + décomposition domaine
Phase 3	Mars - Avril	Hybride + optimisations
Phase 4	Avril	Benchmarks + analyse
Rendu	~Mai 2025	Rapport

Outils et Environnement

- OpenMP 5.0, MPI 4.0
- Machines multi-coeurs locales
- Profiling : gprof, perf, Intel VTune

Métriques importantes :

1. **Speedup** : $S_p = \frac{T_1}{T_p}$ (temps séquentiel / temps parallèle)
2. **Efficacité** : $E_p = \frac{S_p}{p}$ (speedup / nombre de processeurs)
3. **Strong scaling** : Nombre de boid constant, augmenter le nombre de processeurs
4. **Weak scaling** : Charge par processeur constante, augmenter la taille du problème
5. **FPS maintenu** : Objectif 60 FPS avec 10000 boids

Benchmarks comparatifs : Séquentiel, OpenMP, MPI, Hybride

- **S1** : Simulateur complet et fonctionnel (3 règles de Reynolds)
- Architecture modulaire et optimisée (SpatialGrid)
- Visualisation fluide (60 FPS @ 500 agents)
- **S2** : Passage à l'échelle via OpenMP/MPI
- Objectif : 10000+ boids temps réel
- Code source disponible : [lien vers GitHub](#)

- Reynolds, C. W. (1987). *Flocks, herds and schools : A distributed behavioral model*. SIGGRAPH '87.
- Reynolds, C. W. - Red3d Boids : <https://www.red3d.com/cwr/boids/>
- SFML Documentation : <https://www.sfml-dev.org/documentation/>
- OpenMP Documentation : <https://www.openmp.org/>
- MPI Standard : <https://www.mpi-forum.org/>
- GoogleTest Framework : <https://google.github.io/googletest/>

Merci de votre attention !

Questions ?